SVKM's NMIMS
Mukesh Patel School of Technology Management & Engineering
Program:B.Tech/MBA TechAI Semester: VI

Course: Advanced topics in Machine Learning

Experiment No.05

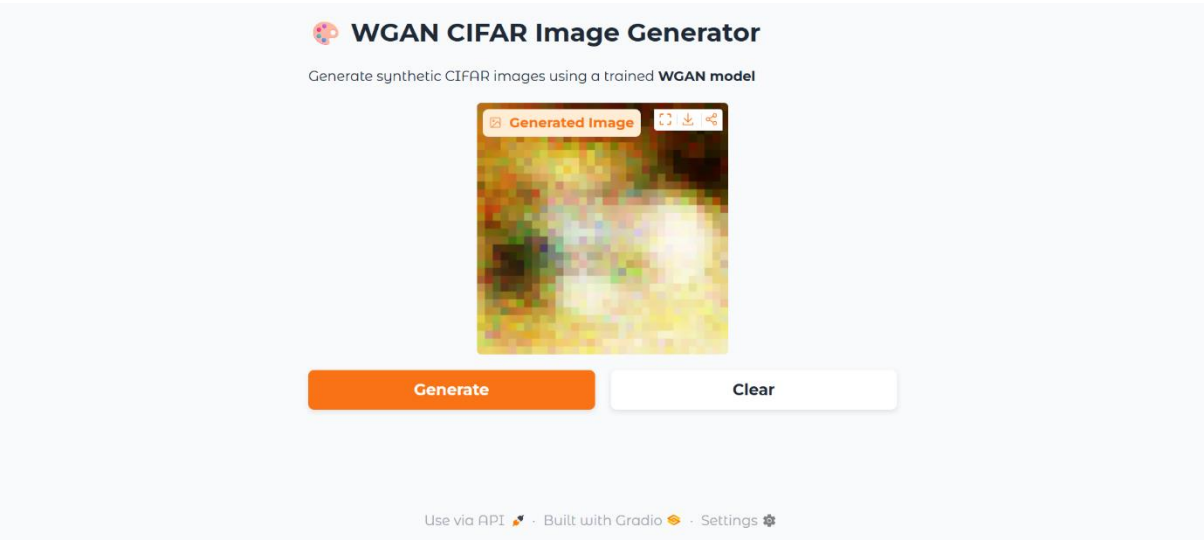
Roll No.: I052	Name: Reya Oberoi
Class : BTech AI	Batch : B2
Date of Experiment: 04-03-26	Date of Submission: 11-03-26
Grade :	

Wasserstein Generative Adversarial Network (WGAN)

Implemented a Wasserstein GAN (WGAN) model for generating synthetic CIFAR images and understand stable GAN training using Wasserstein loss. Also, deployed the trained model using a Gradio-based frontend interface.

Github repository:
https://github.com/reyaoberoi/Reya_ATML_WGAN_CIFAR_Lab5.git

Final Output (Frontend Result)



Interface

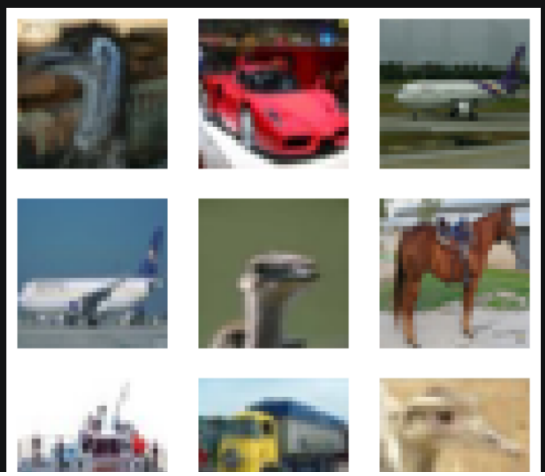
The interface allows users to generate synthetic CIFAR-like images using a trained WGAN model. On clicking **Generate**, a new image is produced from random noise using the generator network.

Notebook Explanation & Training Process

- Loaded CIFAR dataset and normalized pixel values to the range [-1, 1]
- Implemented Generator using fully connected layers with ReLU and Tanh activation
- Designed Critic with Linear + LeakyReLU layers to output real-valued scores (no sigmoid)
- Used random noise vector of size 100 as input to generate 32×32×3 images
- Applied Wasserstein loss for improved training stability
- Trained the critic multiple times per generator update for better gradient estimation
- Enforced Lipschitz constraint using weight clipping
- Generator output scaled using Tanh to match normalized input distribution
- Training showed more stable convergence and reduced mode collapse compared to traditional GANs

```
dataiter = iter(loader)
images, labels = next(dataiter)

fig = plt.figure(figsize=(6,6))

for i in range(9):
    ax = fig.add_subplot(3,3,i+1)
    img = images[i] / 2 + 0.5
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg,(1,2,0)))
    plt.axis("off")
```

[12]



Sample images from CIFAR dataset used for training

```
print(G)
print(C)
```

[13]

```
Generator(
  (net): Sequential(
    (0): Linear(in_features=100, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=1024, bias=True)
    (5): ReLU()
    (6): Linear(in_features=1024, out_features=3072, bias=True)
    (7): Tanh()
  )
)
Critic(
  (net): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=3072, out_features=512, bias=True)
    (2): LeakyReLU(negative_slope=0.2)
    (3): Linear(in_features=512, out_features=256, bias=True)
    (4): LeakyReLU(negative_slope=0.2)
    (5): Linear(in_features=256, out_features=1, bias=True)
  )
)
```

Generator and Critic architecture used in WGAN

Hyperparameters

```
z_dim = 100
lr = 0.00005
batch_size = 64
n_critic = 5
clip_value = 0.01
epochs = 50
```

[7]

Optimizers

```
opt_G = torch.optim.RMSprop(G.parameters(), lr=lr)
opt_C = torch.optim.RMSprop(C.parameters(), lr=lr)
```

[8]

Hyperparameters and Optimizers used

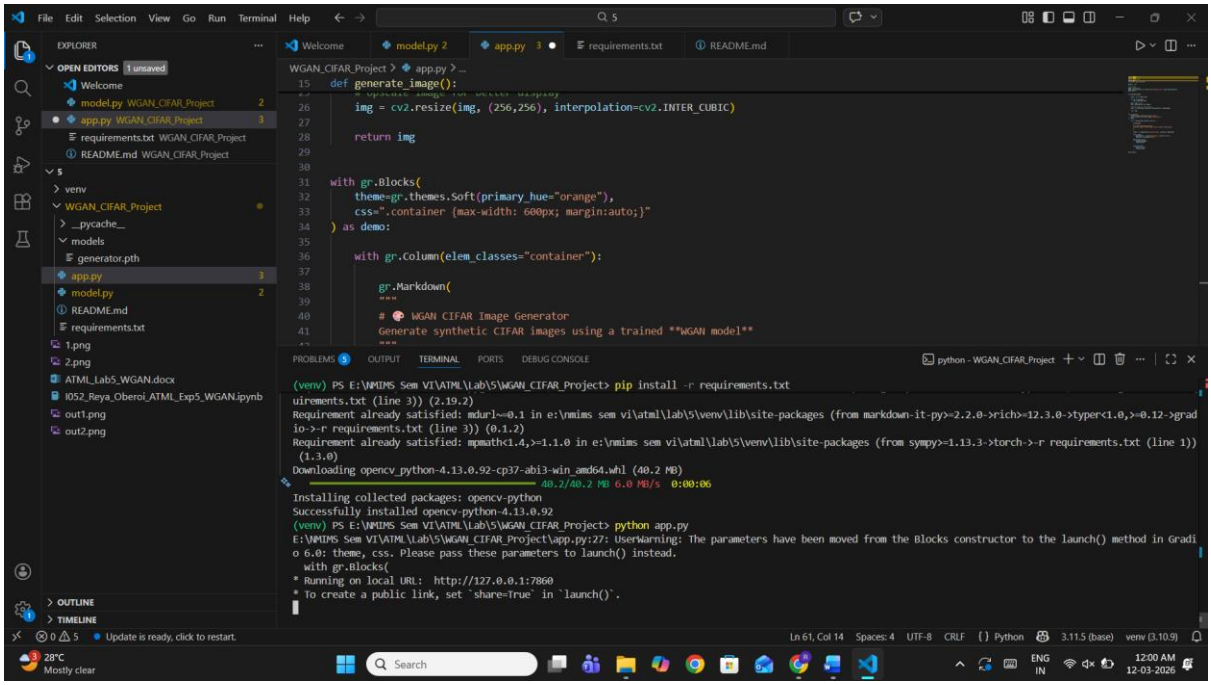
```
fig = plt.figure(figsize=(6,6))

for i in range(16):
    ax = fig.add_subplot(4,4,i+1)
    img = fake[i] / 2 + 0.5
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg,(1,2,0)))
    plt.axis("off")
```

Generated images from trained WGAN model

Trained a WGAN on the CIFAR-10 dataset using a generator and a critic network. During training, the critic learned to distinguish real images from generated ones while the generator improved its ability to create realistic images from random noise. The results show that the model successfully learns the data distribution and can generate synthetic images similar to CIFAR-10 samples.

Integrating with frontend

The experiment was implemented using modular code with separate files such as model.py for defining the Generator and Critic architectures, and app.py for deploying the trained model using Gradio. The frontend allows users to generate images interactively using the trained WGAN model. The complete experiment, including training notebook, model files, and deployment code, was version controlled and uploaded to GitHub for reproducibility and easy access.



Uploading on Github

```
MINGW64:/e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main|M
ERGING)
$ git init
Reinitialized existing Git repository in E:/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project/.
git/

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main|MERGING)
$ git add .

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main|MERGING)
$

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main|MERGING)
$ git commit -m "Initial WGAN CIFAR implementation"
[main da76cfd] Initial WGAN CIFAR implementation

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main)
$ git remote add origin https://github.com/reyaoberoi/Reya_ATML_WGAN_CIFAR_Lab5.git
error: remote origin already exists.

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main)
$ git branch -M main

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main)
$ git push -u origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (13/13), 13.54 MiB | 2.90 MiB/s, done.
Total 13 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/reyaoberoi/Reya_ATML_WGAN_CIFAR_Lab5.git
   4ede2ed..da76cfd  main -> main
branch 'main' set up to track 'origin/main'.

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main)
$ git reset --soft HEAD~1

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main)
$ git add .

RD@LAPTOP-90QF8NIE MINGW64 /e/NMIMS Sem VI/ATML/Lab/5/WGAN_CIFAR_Project (main)
$ git commit -m "Added output screenshots to README"
[main 911dba4] Added output screenshots to README
 4 files changed, 26 insertions(+), 1 deletion(-)
 create mode 100644 LICENSE
 create mode 100644 screenshots/out1.png
 create mode 100644 screenshots/out2.png
```

Conclusion:

The WGAN model successfully generates CIFAR-like images with stable training behavior. While image quality is slightly blurred, the model effectively learns data distribution, demonstrating the advantage of Wasserstein loss over traditional GANs.